

AI Test Case Generator from User Stories

Automating Software Test Case Generation Using Artificial Intelligence

FINAL YEAR PROJECT

SOFTWARE ENGINEERING



Speaker Notes: Welcome the examiners and introduce the project.

This system addresses a critical bottleneck in software testing by leveraging AI to automatically generate test cases from plain-language user stories. The presentation covers the problem, architecture, technology stack, modules, results, and future roadmap.

The Problem with Manual Testing

Time-Consuming

Manual test case creation consumes significant developer and QA effort, slowing release cycles and increasing project costs.

Human Error

Inconsistent documentation and overlooked scenarios reduce test coverage and introduce defects into production.

No Consistency

Without standardization, test quality varies between team members, making reviews and audits difficult.

Intelligent Automation Gap

Existing tools lack AI-driven generation capable of interpreting user stories and producing comprehensive test scenarios.



Speaker Notes: Emphasize that manual test case writing is one of the most time-intensive tasks in the SDLC. Studies show QA teams can spend up to 40% of sprint time on test documentation alone. Human error in this phase directly impacts defect leakage to production. Current tools like TestRail or Zephyr help organize tests but do not generate them intelligently from requirements.

Project Objectives



→ Automate Test Generation

Generate positive, negative, and edge test cases directly from user stories using AI.

→ Improve Software Quality

Ensure comprehensive scenario coverage to reduce defect leakage into production.

→ Reduce Tester Effort

Minimize manual documentation workload, freeing QA teams for higher-value tasks.

→ Support Gherkin Format

Output structured Given-When-Then scenarios ready for BDD frameworks like Cucumber.

- ❑ **Speaker Notes:** These four objectives guided every design decision. The system accepts plain English user stories and outputs structured test cases in Gherkin (Given-When-Then) format, which integrates directly with BDD tools like Cucumber and SpecFlow. The AI model is prompted to always produce three categories: positive paths (happy flow), negative paths (error handling), and edge cases (boundary conditions).

Existing System vs Proposed System

Existing Approach

Manual Writing

Testers write every scenario by hand — slow and error-prone.

High Effort

Significant time investment per user story with no reuse.

Inconsistent Quality

Output quality depends entirely on individual tester expertise.

Limited Coverage

Edge cases and negative paths are frequently overlooked.

Proposed AI System

AI-Powered Generation

Ollama with Llama 3 and Mistral interprets user stories automatically.

Automated Scenarios

Full test suites created in seconds, not hours.

Coverage Analysis

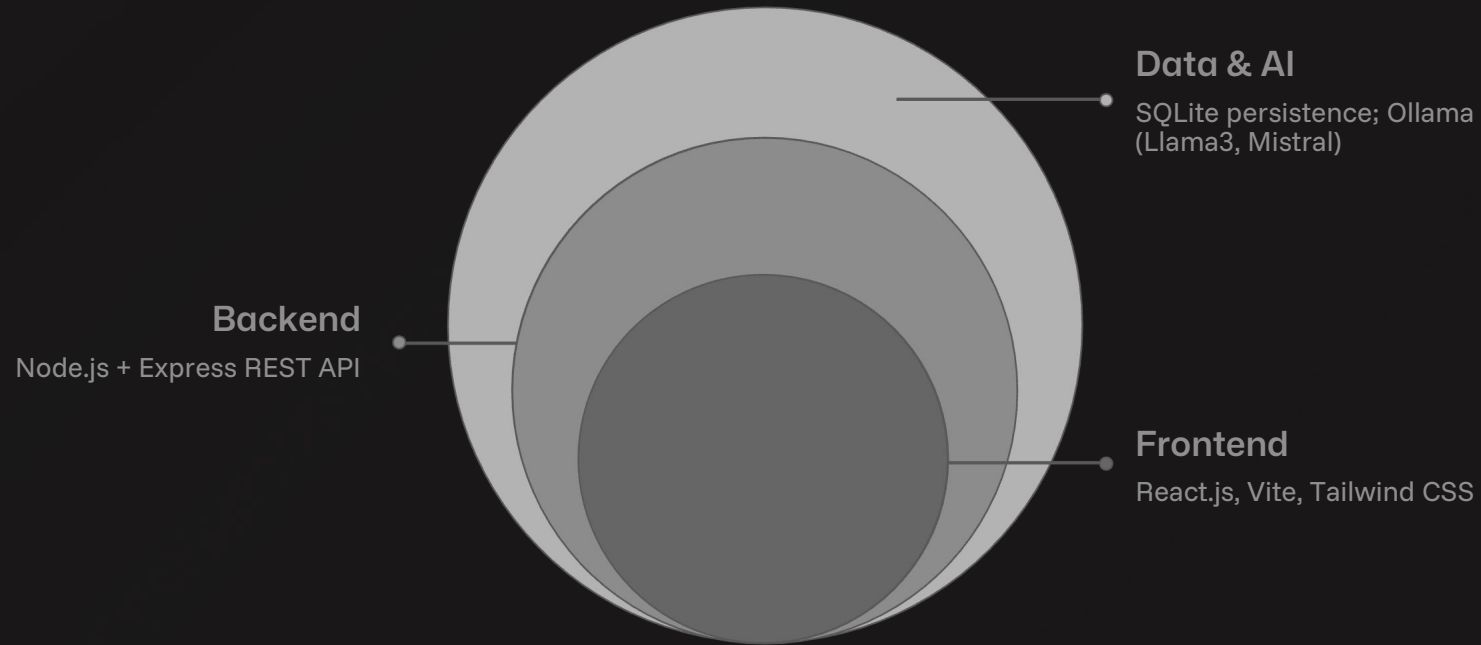
Built-in scoring ensures comprehensive scenario validation.

Export & History

Export as PDF, CSV, or JSON with full session tracking.

📝 **Speaker Notes:** This comparison highlights the shift from a manual, human-dependent process to an automated, AI-driven workflow. The proposed system eliminates the inconsistency and time overhead of the existing approach. Coverage analysis is a key differentiator — the AI evaluates generated cases against the original user story to ensure no critical path is missed. Export options support integration with existing QA toolchains.

System Architecture



The four-tier architecture ensures clean separation of concerns. The React frontend communicates with a Node.js/Express REST API, which orchestrates AI inference via Ollama and persists all data in SQLite. Each layer is independently deployable, supporting future cloud migration.

❏ **Speaker Notes:** Walk through each layer from top to bottom. The frontend is a single-page React app built with Vite for fast loading. The backend exposes RESTful endpoints that handle business logic and AI communication. Ollama runs locally as a model server, calling Llama 3 or Mistral to interpret user stories. SQLite was chosen for simplicity in a academic project context but can be replaced with PostgreSQL for production. All inter-layer communication uses JSON over HTTP.

Technology Stack

Frontend

React.js · Vite · Tailwind CSS

Backend

Node.js · Express.js

Database

SQLite

AI Engine

Ollama · Llama 3 · Mistral

Dev Tools

Git · VS Code

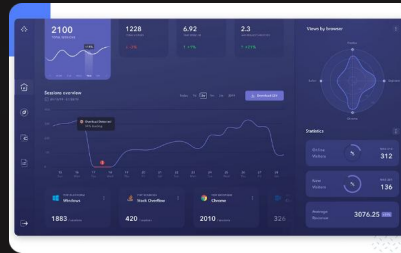
📄 **Speaker Notes:** React.js with Vite provides a fast, modern frontend with hot module replacement for development. Tailwind CSS enables rapid UI styling without custom CSS files. Node.js and Express form a lightweight but scalable backend. Ollama allows running large language models locally without cloud dependencies — critical for data privacy. Llama 3 and Mistral were selected for their strong natural language understanding and instruction-following capabilities. Git and VS Code are standard development tools for version control and coding.

Core Modules



User Authentication

Secure registration, login, and role-based access control protect user data and session integrity throughout the application.



Dashboard Module

User story input, one-click generation, coverage statistics, confidence scores, export options, and full history tracking in one view.

```
import io.cucumber.java.en.When;
import io.cucumber.java.en.Then;
import static org.junit.Assert.*;

public class LoginSteps {

    private WebDriver driver;

    @Given("the user is on the login page")
    public void user_is_on_login_page() {
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");
        driver = new ChromeDriver();
        driver.get("http://example.com/login");
    }

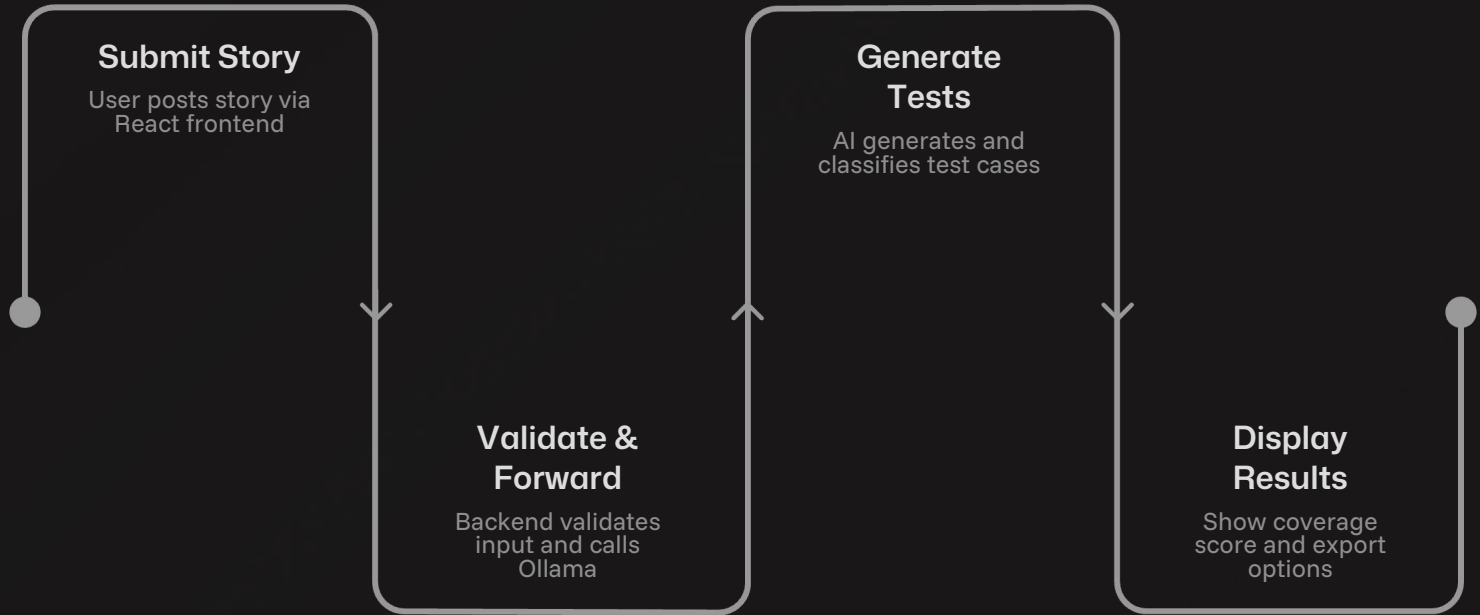
    @When("the user enters valid credentials")
    public void user_enters_valid_credentials() {
        WebElement usernameInput = driver.findElement(By.id("username"));
        usernameInput.sendKeys("user@example.com");
        WebElement passwordInput = driver.findElement(By.id("password"));
        passwordInput.sendKeys("password123");
        WebElement loginButton = driver.findElement(By.id("login"));
        loginButton.click();
    }
}
```

Test Generation Engine

Produces positive, negative, and edge test cases in structured Gherkin Given-When-Then format, ready for BDD frameworks.

- 📄 **Speaker Notes:** The authentication module uses hashed passwords and JWT tokens for session management. The dashboard is the primary user interface — users paste a user story, click Generate, and instantly see test cases with a confidence score indicating AI certainty. The generation engine prompts the AI model to categorize outputs into positive (expected behavior), negative (error conditions), and edge cases (boundary values). All results are saved to the database for history tracking.

Working Process & Test Case Output



The end-to-end pipeline transforms a plain-language user story into a fully structured test suite in seconds, with built-in coverage analysis and multi-format export.

Sample Gherkin Output

```
Feature: User Login
  Scenario: Valid credentials
    Given the user is on the login page
    When they enter valid credentials
    Then access is granted

  Scenario: Invalid password
    Given the user is on the login page
    When they enter an incorrect password
    Then an error is displayed
```

Test Case Categories

Positive Cases
Valid inputs and expected happy-path behavior.

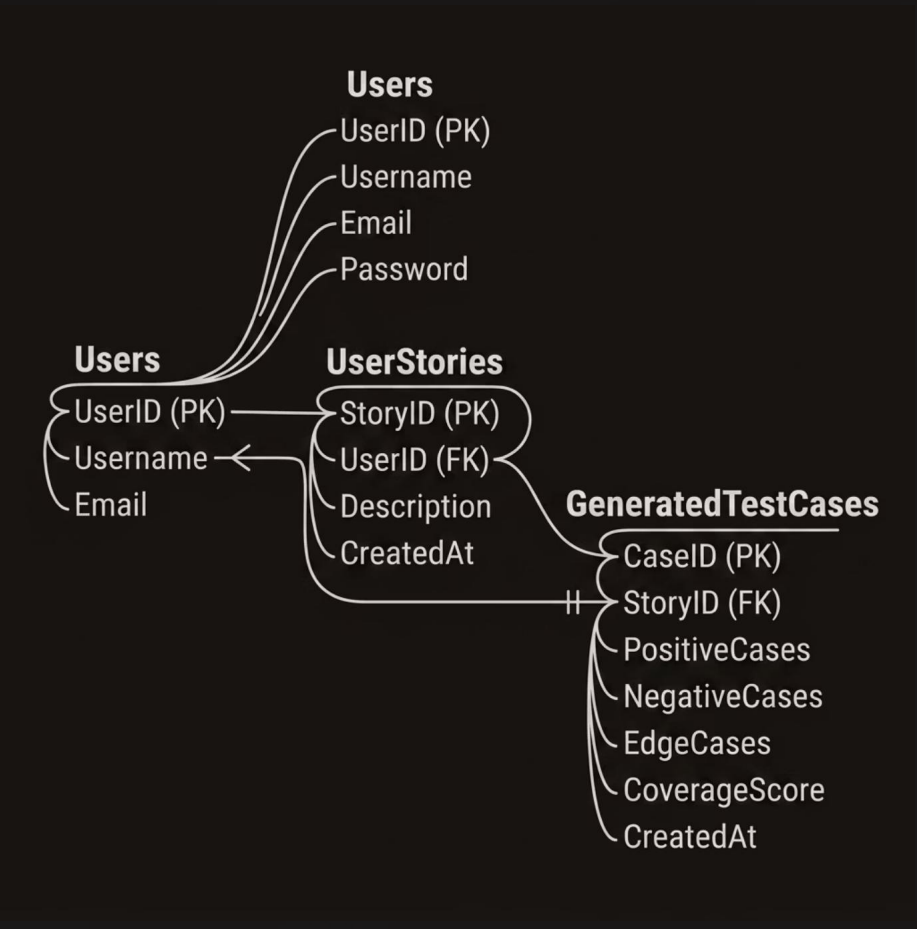
Negative Cases
Invalid inputs and error-condition handling.

Edge Cases
Boundary values and unusual scenarios.

Speaker Notes: The working process diagram shows the linear flow from user input to AI output. The sample Gherkin demonstrates the structured format the system produces automatically. Positive cases cover the expected success path, negative cases handle invalid inputs like wrong passwords, and edge cases address boundary conditions such as empty fields or maximum-length inputs. The confidence score displayed alongside results indicates how certain the AI model is about its generated scenarios.

Database Design & API Endpoints

ER Diagram



REST API Endpoints

Method	Endpoint	Purpose
GET	/api/health	System health check
POST	/api/generate	Generate test cases
GET	/api/history	List all sessions
GET	/api/history/:id	Get session detail
DELETE	/api/history/:id	Delete a session

Speaker Notes: The database has three core tables with foreign key relationships. Users store authentication credentials. UserStories captures each submitted story linked to its author. GeneratedTestCases stores the AI output with all three case categories and a coverage score. The REST API follows standard conventions — GET for retrieval, POST for creation, DELETE for removal. The /api/generate endpoint accepts a user story string and returns the full test case set with confidence metrics.

Key Features, Results & Future Roadmap



Fast AI Generation

Test suites created in seconds via Llama 3 and Mistral.



Coverage Analysis

Automated scoring validates scenario completeness.



Multi-Format Export

Export as PDF, CSV, or JSON for any toolchain.



History Management

Full session tracking with review and replay.

Results: The system successfully generates structured test cases from user stories with measurable coverage scores. **Future enhancements** include multi-model AI support, cloud deployment, Jira integration, test execution automation, multi-language support, and CI/CD pipeline integration.

The AI Test Case Generator automates software testing documentation by generating high-quality test cases from user stories using AI — reducing manual effort and improving software quality across the development lifecycle.

Thank You

Questions & Answers

- 📄 **Speaker Notes:** Summarize the key achievements: automated generation, coverage analysis, Gherkin output, and multi-format export. Demo the live system if available — show login, dashboard input, generation output, and export. For future work, emphasize that the architecture supports adding new AI models via Ollama, and cloud deployment would enable team collaboration. Jira and CI/CD integration would close the loop between requirements and automated test execution. Thank the examiners and invite questions.